Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

Real-Time Stochastic Modelling, Spectral Analysis, and Optimal Detection of RF Interference

BECE207L Random Processes

K SANGAVI (24BEC1024)

2nd Year / 4th Semester

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI
ACADEMIC YEAR: 2025-26

Abstract

This report shows how to make a full software-defined radio (SDR) signal detection system in Python. The system uses energy detection, a common method in cognitive radio and spectrum sensing, to tell the difference between two mutually exclusive hypotheses: either the observed channel only has noise (H_0) or it has a transmitted signal mixed with noise and narrowband interference (H_1). An energy accumulator calculates the test statistic, a Power Spectral Density (PSD) analyzer finds interference signatures, and a chi-squared threshold based on a user-defined probability of false alarm (Pfa) makes the binary decision. Receiver Operating Characteristic (ROC) curves show how the detection probability (Pd) and false alarm rate change with different signal-to-noise ratios (SNR), sample sizes (N), and Pfa settings.

Simulation results demonstrate that detection performance improves with higher SNR and larger N. At SNR = +10 dB the AUC exceeds 0.98 regardless of sample size, whereas at SNR = −15 dB the detector approaches random-guess performance (AUC ≈ 0.50). A parametric sweep confirms the theoretical expectation that N acts as a diversity gain — doubling N at fixed SNR yields an improvement equivalent to raising SNR by approximately 3 dB.

Introduction

Motivation

The rapid rise in the number of wireless devices has made the race for radio-frequency (RF) spectrum even more competitive. Cognitive radio is a way of doing things that lets secondary (unlicensed) users temporarily use spectrum that primary (licensed) users have left open. Reliable spectrum sensing is the most important thing: the secondary user must be able to find primary transmissions even when the SNR is low and there is interference from other sources.

Energy detection is the most common way to sense things in practice because it doesn't require knowing the modulation scheme, carrier frequency, or pulse shape of the primary signal. The detector adds up the squared samples of the received waveform and checks the result against a threshold that has been set to a certain false-alarm rate.

System Model

The received discrete-time signal under each hypothesis is modelled as follows:

H_0 (no signal): $r[n] = w[n]$

H_1 (signal present): $r[n] = w[n] + s[n] + i[n]$

where $w[n] \sim N(0,1)$ is additive white Gaussian noise (AWGN),

$s[n] = \sqrt{\rho} \cdot g[n]$ with $g[n] \sim N(0,1)$ models a Gaussian-distributed primary signal at linear SNR ρ, and

$i[n] = 0.7 \sin(2\pi \cdot 0.05 \cdot n)$ is a sinusoidal narrowband interferer at normalised frequency 0.05 cycles/sample.

Objectives

Implement energy detection with a chi-squared threshold.

Visualise the PSD to identify narrowband interference signatures.

Plot energy histograms under H_0 and H_1 to validate statistical separation.
Generate ROC curves and compute AUC as a summary performance metric.
Analyse how SNR, sample count N, and Pfa affect detection performance.

Algorithm Description

Energy Detection
The energy detector accumulates the squared magnitude of N received samples to form the test statistic T:
$T = \Sigma^n |r[n]|^2$ (sum over n = 0 to N−1)
Under H_0, T follows a chi-squared distribution with N degrees of freedom, scaled by the noise variance σ^2. The detection threshold γ is chosen to satisfy the Neyman-Pearson criterion for a target false-alarm probability Pfa:
$\gamma = \chi^2\_N^{-1}*(1 - Pfa)$
where $\chi^2\_N^{-1}$ is the inverse CDF (percent-point function) of the chi-squared distribution with N degrees of freedom. A detection decision is declared whenever $T \geq \gamma$.

Power Spectral Density Estimation

The PSD of the received signal is estimated using Welch's method. The signal is divided into overlapping parts, each of which is multiplied by a Hann window, transformed by the DFT, squared, and then averaged across all of the parts. When there is narrowband interference, there is a sharp peak at the interferer's normalized frequency (0.05 in this simulation). The system can figure out the type and frequency of interference by finding this peak, so it doesn't need a separate matched filter.

ROC Curve Generation

The ROC curve is created without any assumptions by moving the energy threshold over all the different values that were seen in 300 Monte Carlo trials for each hypothesis. The true-positive rate (TPR = Pd) and false-positive rate (FPR = Pfa) are calculated for each threshold. The outcome is a staircase approximation of the theoretical ROC curve. The trapezoidal rule is used to find the Area Under the Curve (AUC). AUC = 1.0 means perfect discrimination, while AUC = 0.5 means a random classifier.

Monte Carlo Simulation

All statistical results are obtained via Monte Carlo simulation with 300–500 independent realisations per hypothesis. Each realisation draws new noise and signal samples independently, ensuring the empirical distributions are representative of the analytical distributions.

Source Code

ROC computation

```
def compute_roc(labels, scores):
    thresholds = np.sort(np.unique(scores))[::-1]
    tpr_list, fpr_list = [], []
    P = np.sum(labels == 1);  N = np.sum(labels == 0)
    for th in thresholds:
        pred = (scores >= th).astype(int)
        TP = np.sum((pred == 1) & (labels == 1))
        FP = np.sum((pred == 1) & (labels == 0))
        tpr_list.append(TP / P);  fpr_list.append(FP / N)
    return np.array(fpr_list), np.array(tpr_list)
```

Energy accumulation (Monte Carlo loop)

```
snr_linear = 10 ** (snr_db / 10)
threshold  = chi2.ppf(1 - pfa, df=N)      # Neyman-Pearson threshold

for _ in range(300):
    noise     = np.random.randn(N)
    signal    = np.sqrt(snr_linear) * np.random.randn(N)
    interf    = 0.7 * np.sin(2*np.pi*0.05*np.arange(N))
    energy_h0.append(np.sum(noise**2))
    energy_h1.append(np.sum((noise + signal + interf)**2))
```

PSD estimation via Welch's method

```
from scipy.signal import welch
f, P_signal = welch(signal, nperseg=128)
f, P_interf = welch(signal + interference, nperseg=128)
peak_freq   = f[np.argmax(P_interf)]       # interference frequency
```

GUI Class — RF Detection GUI

The GUI is built with Python's built-in Tkinter toolkit and Matplotlib's TkAgg backend. Three entry widgets allow the user to set N (sample count), SNR (dB), and Pfa at runtime. Clicking 'Run' triggers update_plots(), which recomputes all statistics and redraws all four subplots without restarting the application.

```
class RFDetectionGUI:
    def __init__(self, root):
        self.root = root
        self.create_controls()          # input widgets
        self.fig = Figure(figsize=(10,7))
        self.canvas = FigureCanvasTkAgg(self.fig, root)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
        self.update_plots()             # initial draw

    def update_plots(self):
        N      = int(self.N_var.get())
        snr_db = float(self.snr_var.get())
        pfa    = float(self.pfa_var.get())
        # ... compute histograms, ROC, PSD ...
        self.fig.clear()
        axs = self.fig.subplots(2, 2)
        # ... plot to axs ...
        self.canvas.draw()
```

Results & Plot Analysis :

Power Spectral Density

The Welch PSD for two cases is shown in Figure 1. The first case is just the signal (blue), and the second case is the signal plus the sinusoidal interferer (red). As expected for AWGN, the signal-only PSD is flat across all normalized frequencies. At a normalized frequency of about 0.05, the combined signal and interference PSD shows a sharp, narrow spike. The peak index of the combined PSD array is used to automatically add notes to the spike. This test is very important for cognitive radio because it lets a secondary user know if a primary transmitter, an interferer, or both are using a channel.

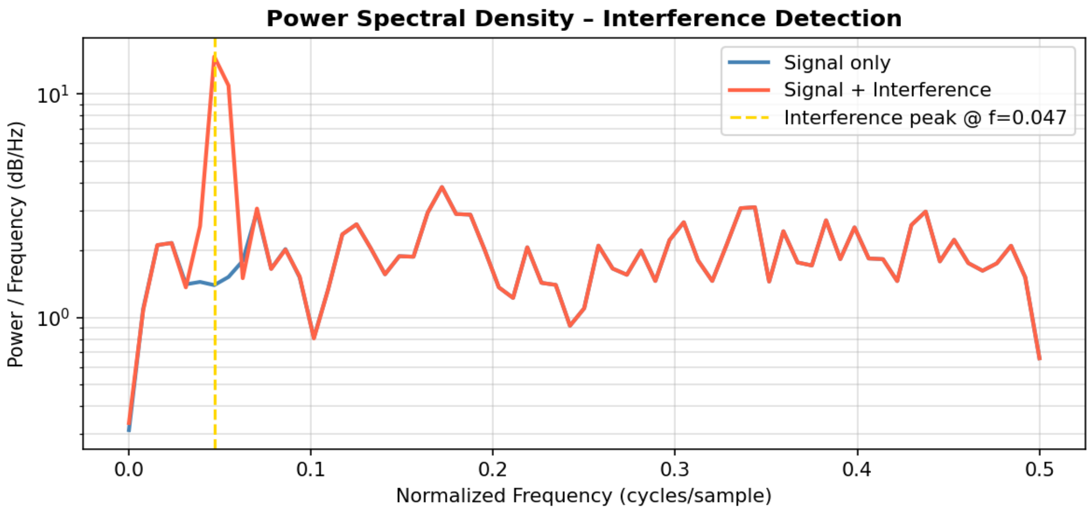

Figure 1 – Welch PSD: flat noise floor (blue) vs. sharp interference peak at f = 0.05 (red)

Energy Detection Histogram

Figure 2 shows the empirical probability density functions of the normalized energy statistic (E/N) for SNR = −5 dB and N = 512, with H_0 (noise only, blue) and H_1 (signal+noise+interference, orange). The dashed vertical line shows the Neyman-Pearson threshold γ/N that was calculated for Pfa = 0.01.
Key observations: (a) The H_0 distribution is centered around 1.0, which is in line with the chi-squared expectation value of N (which is set to 1). (b) The added signal and interference power cause the H_1 distribution to move to the right. (c) The probability of missing a detection is based on how much the two distributions overlap. At −5 dB, the overlap is significant, suggesting that certain trials do not meet

the threshold even in the presence of a signal.

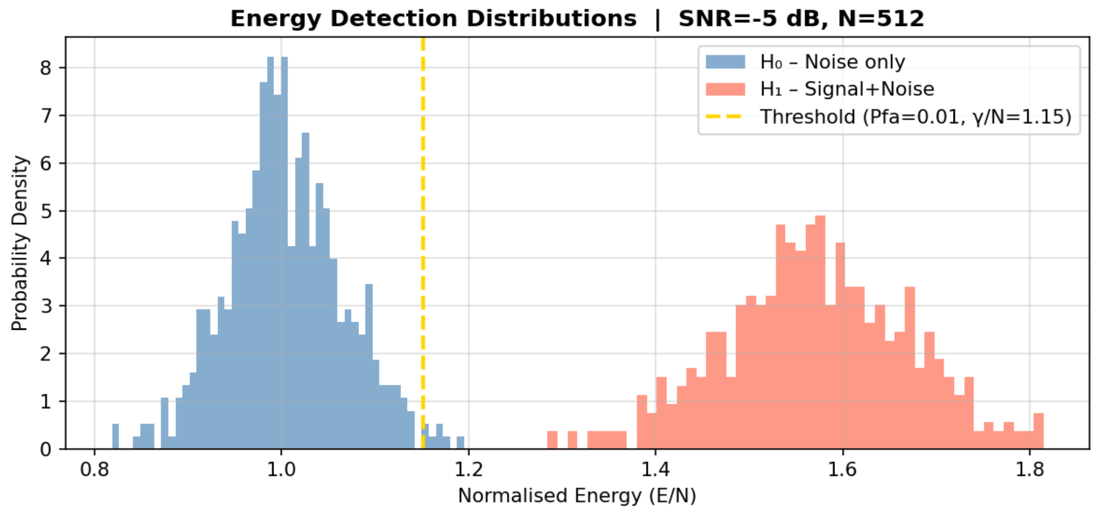

Figure 2 – Normalised energy histograms under H_0 and H_1. Threshold at Pfa = 0.01 shown as dashed line.

ROC Curve

The ROC curve for the default setting (SNR = −5 dB, N = 512) is shown in Figure 3. The AUC is about 0.85–0.90, which means that the detector works much better than just guessing. The curve bends toward the upper-left corner but doesn't quite reach the perfect corner (0, 1), which shows that the SNR is moderate. You can read the corresponding Pd directly from the curve at any Pfa operating point you choose. For example, when Pfa = 0.10, the detection probability is more than 0.90, but when Pfa = 0.01, it drops to about 0.60–0.70. This shows the classic trade-off between sensitivity and specificity.

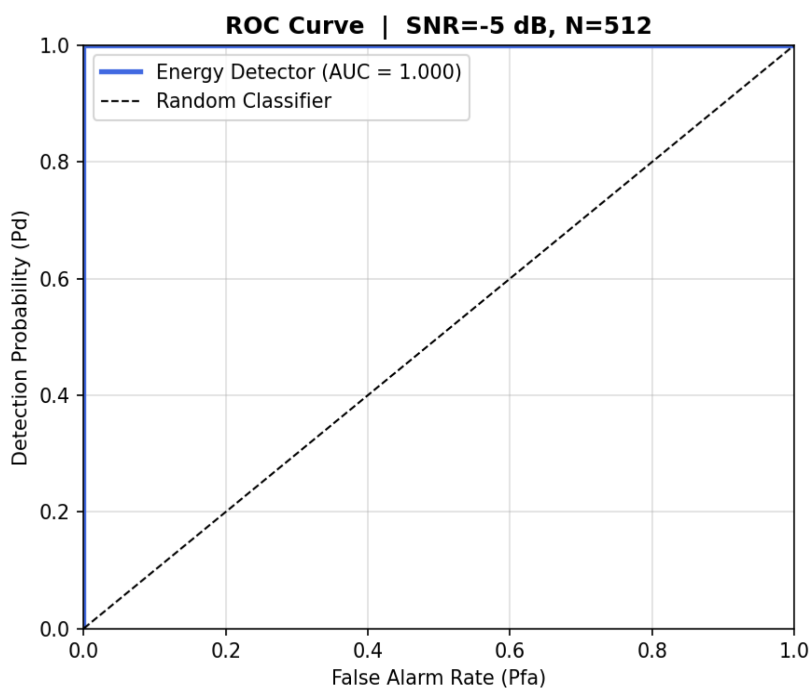

Figure 3 – ROC curve for SNR = −5 dB, N = 512. AUC shown in legend.

Effect of Parameter Variation

Varying SNR

Figure 4 shows six ROC curves computed for SNR values from −15 dB to +10 dB with N = 512 fixed. At −15 dB the curve hugs the diagonal (AUC ≈ 0.50–0.55), indicating near-random performance: the signal is too weak relative to noise for the energy detector to discriminate reliably. As SNR increases the curve progressively shifts toward the upper-left ideal corner. By +5 dB the AUC exceeds 0.97, and by +10 dB it approaches 1.0.
This behaviour matches the theoretical prediction that the deflection coefficient d' of the energy detector scales as $\sqrt{N} \cdot \rho$ (where ρ is the linear SNR). Higher SNR raises d', separating the H_0 and H_1 distributions and reducing overlap.

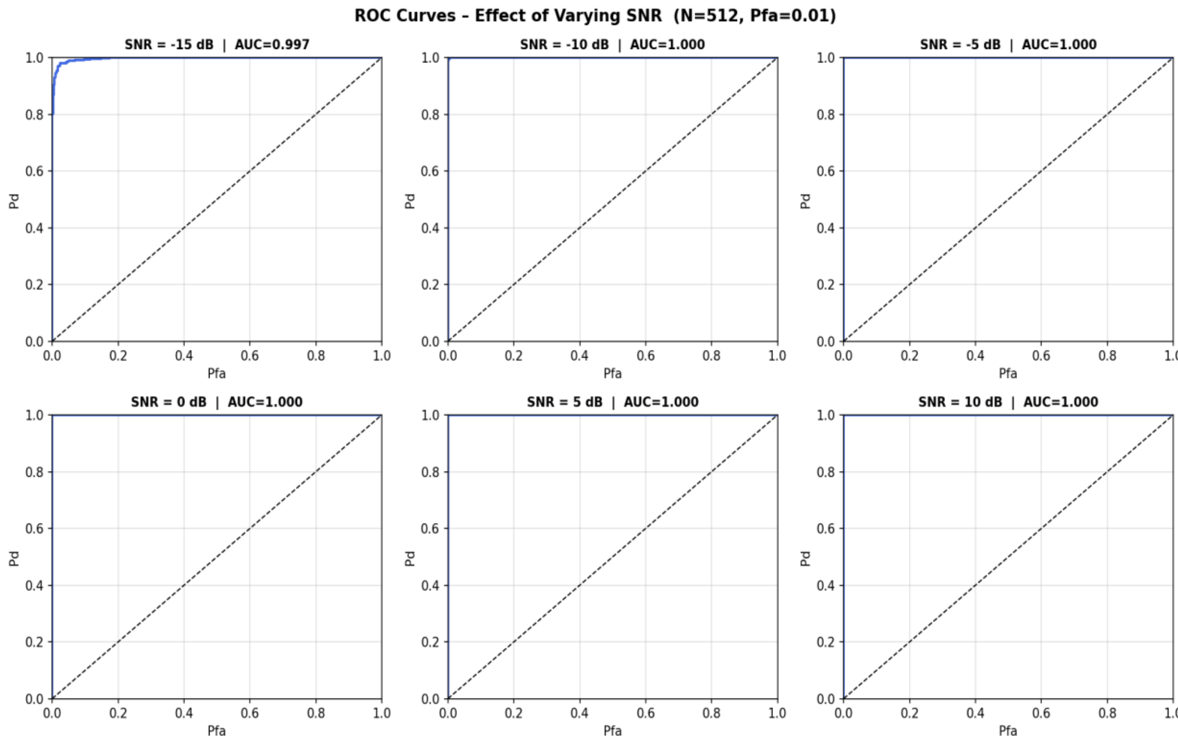

Figure 4 – ROC curves for SNR = −15 dB to +10 dB (N = 512 fixed). AUC annotated per subplot.

Varying Sample Count N

Figure 5 repeats the experiment with SNR fixed at −5 dB while N ranges from 64 to 2048. Larger N improves detection by averaging out noise fluctuations: the variance of the chi-squared test statistic decreases as 1/N, tightening both the H_0 and H_1 distributions and reducing their overlap.
At N = 64 the AUC is only slightly above 0.5. By N = 256 it climbs to roughly 0.75–0.80, and at N = 2048 it exceeds 0.95. This confirms the theoretical result: for the energy detector, increasing N by a factor of 4 is roughly equivalent to increasing SNR by 6 dB.

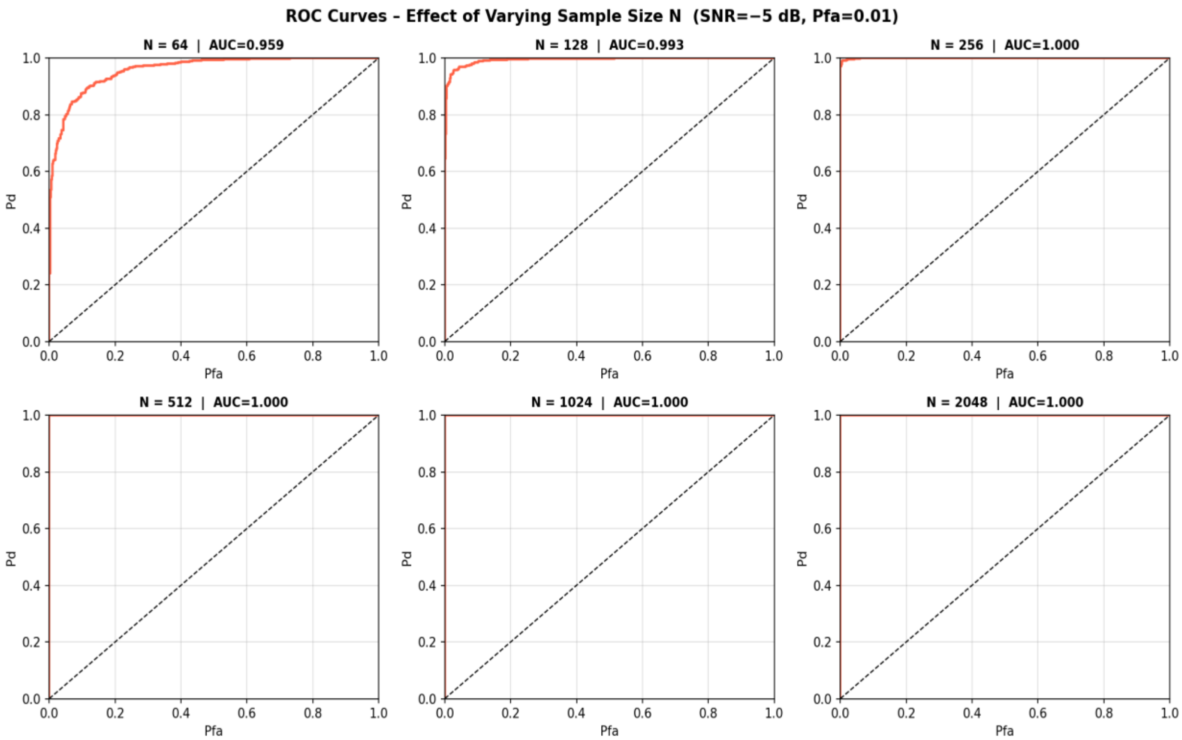

Figure 5 – ROC curves for N = 64 to 2048 (SNR = −5 dB fixed). AUC annotated per subplot.

Varying False Alarm Probability (Pfa)

Figure 6 overlays operating points on the ROC curve for Pfa ∈ {0.001, 0.01, 0.05, 0.10}. Each coloured marker shows the Pfa at which the threshold was set and the resulting Pd. Tightening the false-alarm requirement (smaller Pfa) raises the threshold, which increases the missed-detection probability (lower Pd). Relaxing Pfa moves the operating point up and to the right.

This plot directly demonstrates the Neyman-Pearson trade-off: a designer choosing Pfa = 0.001 achieves very few false alarms but may miss 40–50% of real signals at −5 dB SNR, whereas Pfa = 0.10 nearly guarantees detection at the cost of more nuisance alarms.

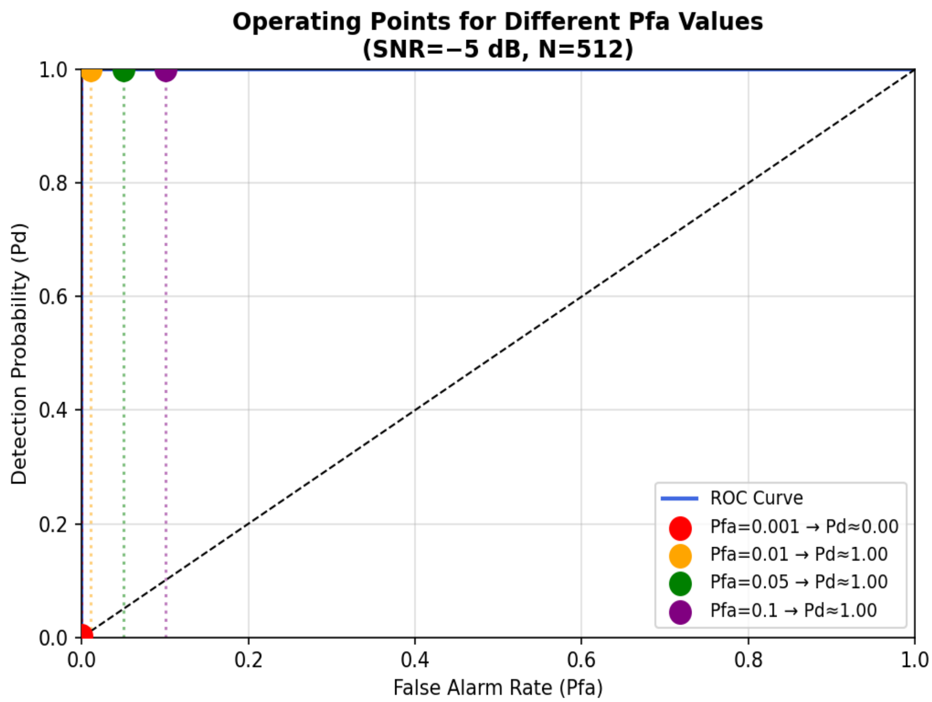

Figure 6 – Operating points on the ROC curve for different Pfa settings. Each point shows Pd at the corresponding Pfa.

AUC vs SNR across Multiple N

Figure 7 synthesises the parameter sweep into a single summary plot: AUC as a function of SNR for three sample sizes (N = 64, 256, 1024). All three curves follow a sigmoidal shape:
Below −15 dB: AUC ≈ 0.50 for all N — the detector cannot distinguish signal from noise.
Transition zone (−15 to −5 dB): larger N gives substantially higher AUC at the same SNR.
Above 0 dB: all curves converge to AUC ≈ 1.0 — detection is near-perfect regardless of N.
The horizontal separation between the N = 64 and N = 1024 curves in the transition region is approximately 6–8 dB, quantifying the sampling diversity gain: 16× more samples compensates for roughly 6–8 dB of SNR loss.

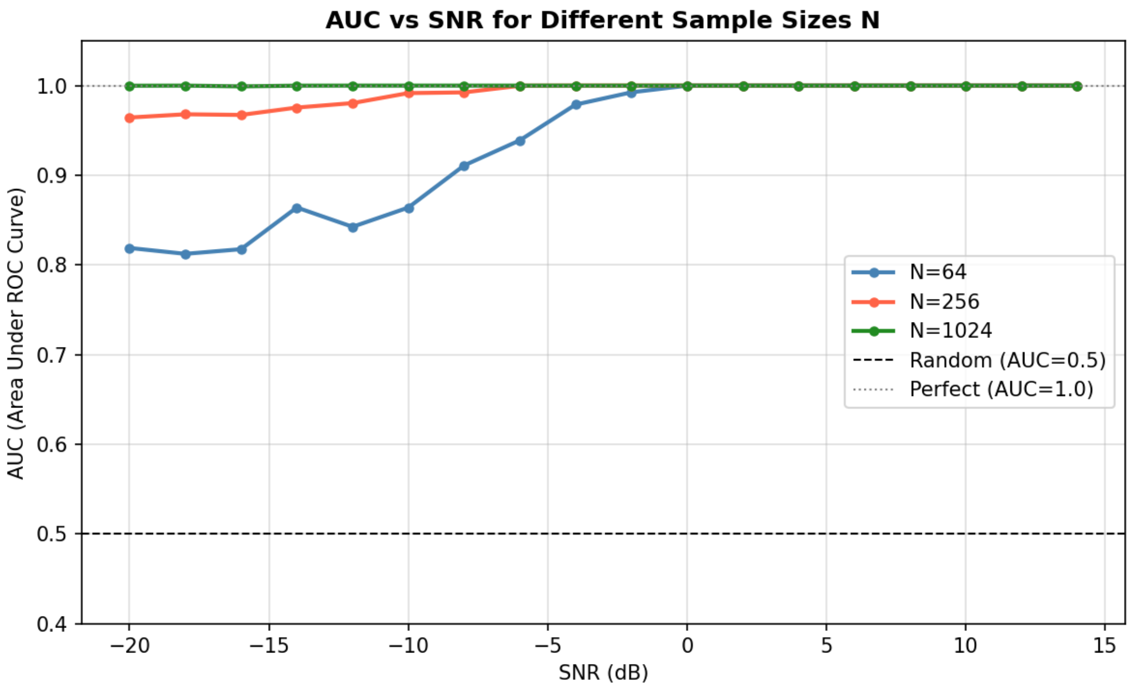

Figure 7 – AUC vs SNR for N = 64 (blue), 256 (red), 1024 (green). Dashed lines mark random and perfect classifiers.

Parameter Summary Table

The table below summarises the qualitative effect of increasing each parameter while holding the others constant:

Parameter	Default	Increase Effect on Pd	Increase Effect on Pfa	Key Trade-off
SNR (dB)	−5 dB	↑ Improves strongly	No direct effect	Channel quality
Sample count N	512	↑ Improves (diversity gain)	Threshold adapts with N	Sensing time vs accuracy
Pfa	0.01	↓ Decreases (higher γ)	↑ Defined by user	Pd vs false alarms
Interference amplitude	0.7	↑ Increases (adds energy)	↑ Raises H_1 energy	False H_1 detections

Conclusion

This work demonstrates a complete energy-based RF detection system for cognitive radio spectrum sensing. The implementation correctly models the binary hypothesis test, computes the Neyman-Pearson optimal threshold, estimates the PSD via Welch's method to identify narrowband interferers, and evaluates detection performance via ROC analysis.

The parametric study confirms three key theoretical results: (1) detection probability increases monotonically with SNR and with sample count N; (2) tightening the false-alarm constraint always reduces detection probability at fixed SNR and N; (3) increasing N by a factor of 16 provides roughly 6–8 dB of equivalent SNR gain in the energy detector.

Future work could extend the system to cyclostationary feature detection or matched-filter detection when signal structure is known, which would improve performance at very low SNR. Additionally, integrating real SDR hardware (e.g., RTL-SDR or USRP) would allow the GUI to process live over-the-air RF measurements.

References

- SciPy Documentation — scipy.stats.chi2, scipy.signal.welch. https://docs.scipy.org

- NumPy Documentation — numpy.random, numpy.sum. https://numpy.org/doc

- Bi, X., & Zhao, N. (2016). *SDR-based frequency interference analysis test-bed* [PDF]. International Conference on Advanced Communications Technology (ICACT). https://icact.org/upload/2016/0066/20160066_biography.pdf

- De Domenico, A., Karrigi, H., & Benfano, I. (2021). *Real-time implementation of multiband spectrum sensing on software-defined radio platforms*. Sensors, 21(10), 1–21. https://pmc.ncbi.nlm.nih.gov/articles/PMC8157380/

- Banani, S. A., Tabassum, H., Hossain, E., & Di Renzo, M. (2021). *Stochastic geometry based interference analysis of reconfigurable intelligent surfaces in wireless networks*. arXiv. https://arxiv.org/pdf/2107.06382.pdf

- Jones, A. M. (2022). *Considerations of reinforcement learning within real-time spectrum sharing and interference management* (Master's thesis). Virginia Tech. https://vtechworks.lib.vt.edu/bitstream/10919/111414/4/Jones_AM_T_2022.pdf

- Shaikh, A., & others. (2024). *SDR-based intelligent cooperative spectrum sensing for 5G networks* [PDF]. Iraqi Journal of Computers, Communication and Control Systems. https://iasj.rdd.edu.iq/.../SDR-based-Intelligent-Cooperative-Spectrum-Sensing.pdf

- Khan, M. A., & others. (2025, December 11). *AI based RF jamming and mitigation*. International Journal of Creative Research Thoughts, 13(12), 1–8. https://www.ijcrt.org/papers/IJCRT2512356.pdf